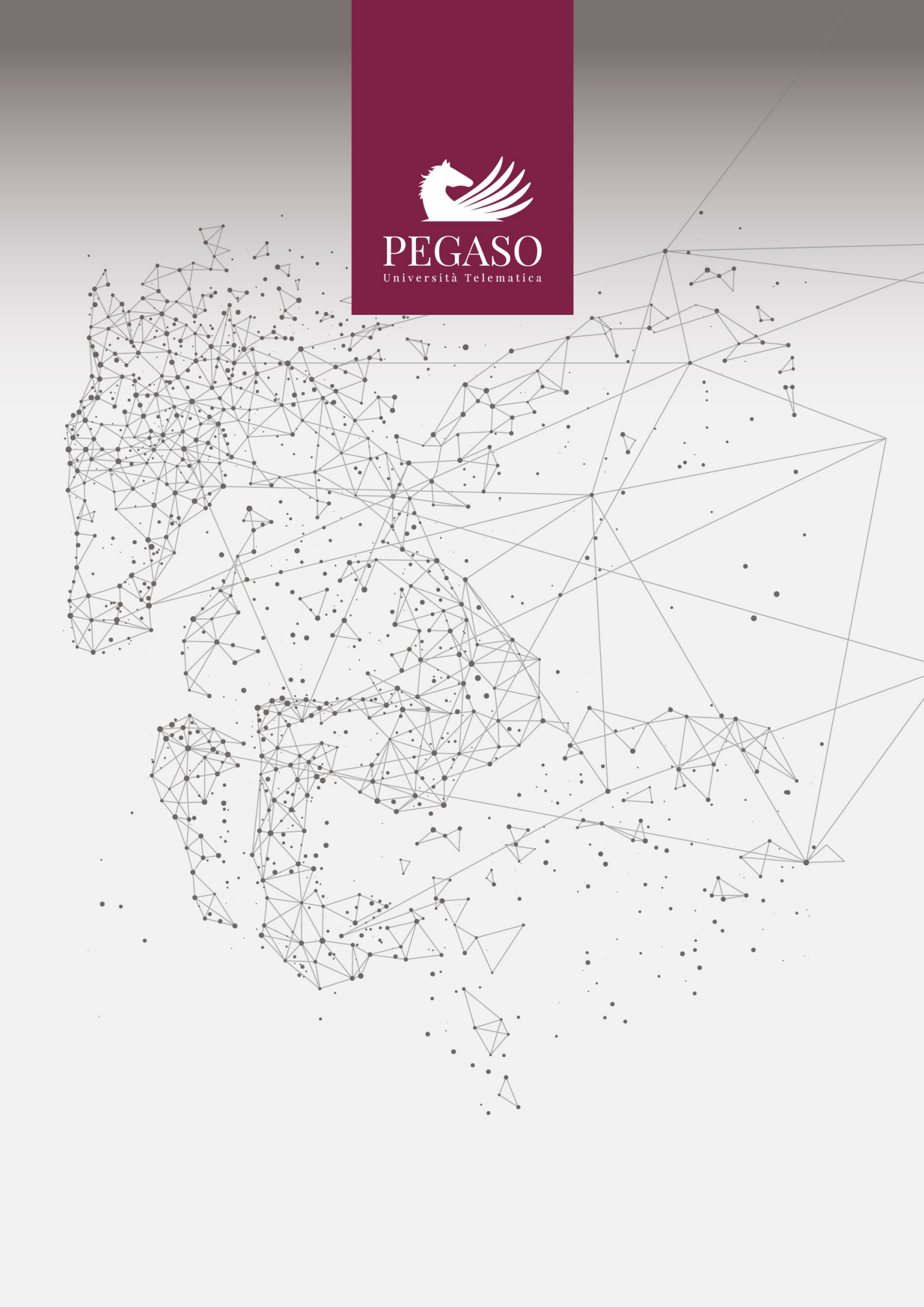

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3
2. CONGIUNZIONI 4
3. FUNZIONI DI AGGREGAZIONE E CLAUSOLA DI RAGGRUPPAMENTO 11
4. QUERY ANNIDATE 14
BIBLIOGRAFIA 15

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si introduce l'operazione di congiunzione JOIN tra due o più tabelle e si approfondiscono tutte le tipologie di congiunzione possibili su database relazionali. Vengono, inoltre, presentate e approfondite le funzioni di aggregazione e la clausola di raggruppamento GROUP BY necessaria per partizionare in gruppi le tuple di una tabella. Infine, viene illustrato il meccanismo delle query annidate, indispensabile per la risoluzione di problemi più complessi o risolvibili per passi successivi.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Congiunzioni

Nei database, i dati da ricercare non sono sempre contenuti in una sola tabella. Per eliminare il problema della ridondanza, le basi di dati vengono, infatti, normalizzate e vengono create N tabelle collegate tra loro attraverso relazioni (associazioni).

L'**operazione di congiunzione JOIN** tra due tabelle prevede la specifica di un criterio che si basa sull'uguaglianza tra la chiave primaria della tabella dominio dell'associazione e la chiave esterna corrispondente della tabella codominio. Questo tipo di join prende il nome di **equi-join**.

Al fine di osservare le relazioni tra le tabelle, in Figura 1, si riporta la struttura del DB (estratto del noto database Northwind di esempio della Microsoft) presente sul sito: https://www.w3schools.com/sql/trysql.asp?filename=trysql_select_all

La struttura prevede associazioni 1:N (uno a molti) per cui è possibile procedere con l'operazione di congiunzione.

Soffermandosi sull'associazione tra la tabella Orders e la tabella Customers è possibile osservare che:

- la tabella Customers ha il campo CustomerID che è chiave primaria;
- la tabella Orders ha il campo CustomerID che è chiave esterna.

In Figura 2, si riportano alcuni esempi di corrispondenza tra i due campi CustomerID nelle due tabelle.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

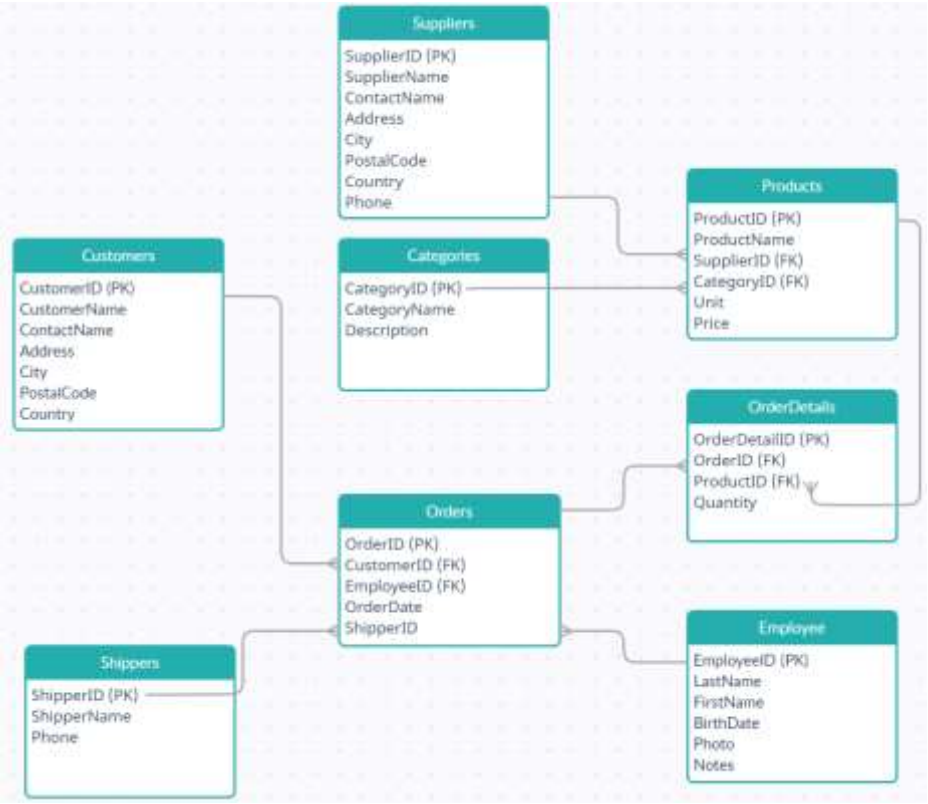


Fig1. Struttura del DB in esame (estratto del noto database Northwind di esempio della Microsoft)

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

OrderID	CustomerID	EmployeeID	OrderDate	ShipperID
10298	37	6	1996-09-05	2
10308	2	7	1996-09-18	3
10309	37	3	1996-09-19	1
10335	37	7	1996-10-22	2
10365	3	3	1996-11-27	2

Fig2. Esempi di corrispondenza tra i due campi CustomerID nelle due tabelle: Customers (in alto) e Orders (in basso)

Procediamo, quindi, con un equi-join sulle tabelle Customers e Orders in base all'associazione vista. Il risultato che si vuole ottenere è l'elenco di identificativo dell'ordine, nome del Customer e data dell'ordine di tutti i Customer italiani (esempio di filtro su equi-join). Segue la query SQL:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
SELECT Orders.OrderID, Customers.CustomerName, Orders.OrderDate
FROM Orders,Customers
WHERE Orders.CustomerID = Customers.CustomerID
AND Customers.Country='Italy'
```

La tabella risultante dalla query precedente risulterà essere:

Result:

Number of Records: 7

OrderID	CustomerName	OrderDate
10275	Magazzini Alimentari Riuniti	1996-08-07
10288	Reggiani Caseifici	1996-08-23
10300	Magazzini Alimentari Riuniti	1996-09-09
10404	Magazzini Alimentari Riuniti	1997-01-03
10422	Franchi S.p.A.	1997-01-22
10428	Reggiani Caseifici	1997-01-28
10443	Reggiani Caseifici	1997-02-12

Per la query appena vista è possibile utilizzare una sintassi diversa in cui la clausola WHERE contiene solo le condizioni (filtri). Tale sintassi prevede l'uso dell'**operatore INNER JOIN** usato come nella query SQL seguente:

```
SELECT Orders.OrderID, Customers.CustomerName, Orders.OrderDate
FROM Orders
INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID
WHERE Customers.Country='Italy';
```

Ovviamente è possibile eseguire la congiunzione tra più di due tabelle "navigando" tra le associazioni. Si supponga di voler eseguire l'operazione di INNER JOIN tra le tabelle Orders, Customers e Shippers (Shippers è collegata ad Orders tramite un'associazione).

La query SQL corrispondente è la seguente:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
SELECT Orders.OrderID, Customers.CustomerName, Shippers.ShipperName
FROM Orders
INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID
INNER JOIN Shippers ON Orders.ShipperID = Shippers.ShipperID
WHERE Customers.Country='Italy';
```

La tabella risultante dalla query precedente risulterà essere:

Result:

Number of Records: 7

OrderID	CustomerName	ShipperName
10275	Magazzini Alimentari Riuniti	Speedy Express
10288	Reggiani Caseifici	Speedy Express
10300	Magazzini Alimentari Riuniti	United Package
10404	Magazzini Alimentari Riuniti	Speedy Express
10422	Franchi S.p.A.	Speedy Express
10428	Reggiani Caseifici	Speedy Express
10443	Reggiani Caseifici	Speedy Express

Per visualizzare i dati di due tabelle anche nel caso di mancate corrispondenze tra le loro righe, è necessario ricorrere all'**operatore OUTER JOIN** nelle forme:

1. LEFT OUTER JOIN (o LEFT JOIN);
2. RIGHT OUTER JOIN (o RIGHT JOIN);
3. FULL OUTER JOIN.

In Figura 3, si riporta la rappresentazione grafica delle tre forme dell'outer join da un punto di vista insiemistico.

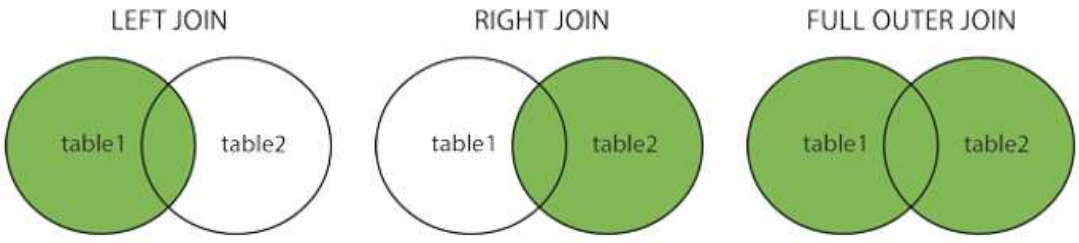


Fig3. Rappresentazione grafica delle tre forme dell'outer join da un punto di vista insiemistico

Il LEFT JOIN restituisce una tabella in cui ci sono:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- tutte le colonne della tabella di sinistra e di quella di destra;
- tutte le tuple della tabella di sinistra e le tuple della tabella di destra per cui c'è una corrispondenza.

Se non c'è una corrispondenza, verrà riportato il valore *null* nelle colonne relative alla tabella di destra. La query SQL seguente esegue il LEFT JOIN:

```
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID
ORDER BY Customers.CustomerName;
```

Un estratto del risultato dell'esecuzione della query è il seguente:

Result:

Number of Records: 213

CustomerName	OrderID
Alfreds Futterkiste	*null*
Ana Trujillo Emparedados y helados	10308
Antonio Moreno Taquería	10365
Around the Horn	10355

Il RIGHT JOIN restituisce una tabella in cui ci sono:

- tutte le colonne della tabella di sinistra e di quella di destra;
- tutte le tuple della tabella di destra e le tuple della tabella di sinistra per cui c'è una corrispondenza.

Se non c'è una corrispondenza, verrà riportato il valore *null* nelle colonne relative alla tabella di sinistra. La query SQL seguente esegue il RIGHT JOIN:

```
SELECT Orders.OrderID, Employees.LastName, Employees.FirstName
FROM Orders
RIGHT JOIN Employees
ON Orders.EmployeeID = Employees.EmployeeID
ORDER BY Orders.OrderID;
```

Un estratto del risultato dell'esecuzione della query è il seguente:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Result:

Number of Records: 197

OrderID	LastName	FirstName
	West	Adam
10248	Buchanan	Steven
10249	Suyama	Michael
10250	Peacock	Margaret

Il FULL OUTER JOIN restituisce una tabella in cui ci sono:

- tutte le colonne della tabella di sinistra e di quella di destra;
- tutte le tuple della tabella di sinistra e della tabella di destra per cui c'è una corrispondenza.

Se non c'è una corrispondenza, verrà riportato il valore *null* nelle colonne relative alla tabella di sinistra o alla tabella di destra.

La query SQL che segue esegue il FULL OUTER JOIN:

```
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
FULL OUTER JOIN Orders ON Customers.CustomerID=Orders.CustomerID
ORDER BY Customers.CustomerName;
```

Un estratto del risultato dell'esecuzione della query è il seguente:

CustomerName	OrderID
Alfreds Futterkiste	*Null*
Ana Trujillo Emparedados y helados	10308
Antonio Moreno Taquería	*Null*

In alcune circostanze è necessario ricorrere anche all'operazione di **auto-congiunzione** (**SELF JOIN**). Il SELF JOIN è la congiunzione tra le colonne di una stessa tabella, in cui i record vengono combinati e aggiunti alle tuple risultato. Mediante questo particolare tipo di congiunzione possiamo creare un vincolo con la stessa tabella.

Nell'esempio successivo, si richiede di ottenere l'elenco dei customer che sono della stessa città. La query SQL che realizza il SELF JOIN è la seguente:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
SELECT A.CustomerName AS CustomerName1, B.CustomerName AS
CustomerName2, A.City
FROM Customers A, Customers B
WHERE A.CustomerID <> B.CustomerID
AND A.City = B.City
ORDER BY A.City;
```

Un estratto del risultato della query precedente di SELF JOIN è:

Result:

Number of Records: 88

CustomerName1	CustomerName2	City
Cactus Comidas para llevar	Océano Atlántico Ltda.	Buenos Aires
Cactus Comidas para llevar	Rancho grande	Buenos Aires
Océano Atlántico Ltda.	Cactus Comidas para llevar	Buenos Aires
Océano Atlántico Ltda.	Rancho grande	Buenos Aires

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Funzioni di aggregazione e clausola di raggruppamento

Il linguaggio SQL fornisce alcune funzioni che possono essere usate nel comando SELECT per determinare:

- la somma, la media aritmetica e la deviazione standard di un insieme di valori (**SUM, AVG** e **STDEV**);
- la cardinalità di un insieme di righe (**COUNT**);
- il massimo e il minimo di un insieme di valori (**MAX** e **MIN**).

SQL permette l'utilizzo contemporaneo di più funzioni di aggregazione anche nel contesto di una singola SELECT.

Di fondamentale importanza è capire la tipologia di risultato della query SQL. Le funzioni di aggregazione restituiscono, infatti, un solo valore riferito ad un insieme di valori, mentre la SELECT normalmente restituisce una tabella. Quando si utilizzano le funzioni di aggregazione è, quindi, richiesto di verificare l'omogeneità del risultato. Si fornisce ora una serie di esempi di applicazione delle funzioni di aggregazione:

1. Esempio di uso di **COUNT**() - trovare il numero dei prodotti nella tabella Products. Sono possibili due forme della query SQL che produrranno lo stesso risultato, cioè 77.
2.

```
SELECT COUNT(ProductID)
FROM Products;
```

```
SELECT COUNT(*)
FROM Products;
```

3. Esempio di uso di **AVG**() - trovare il prezzo medio di tutti i prodotti nella tabella Products. Il risultato sarà 28,87.

```
SELECT AVG(Price)
FROM Products;
```

4. Esempio di uso di **SUM**() - trovare la somma del campo Quantità nella tabella OrderDetails. Il risultato sarà 12.743.

```
SELECT SUM(Quantity)
FROM OrderDetails;
```

5. Esempio di uso di **MAX**() - trovare il prezzo massimo tra tutti i prodotti nella tabella Products. Il risultato sarà 263,50.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
SELECT MAX(Price)
FROM Products;
```

6. Esempio di uso di **STDEV**() - trovare la deviazione standard del prezzo dei prodotti nella tabella Products. Il risultato sarà 33,82.

```
SELECT STDEV(Price)
FROM Products;
```

Il linguaggio SQL permette di **partizionare** in gruppi le righe di una tabella utilizzando la clausola **GROUP BY.** A tali gruppi può essere applicata una qualsiasi delle funzioni di aggregazione.

Per esempio, si vuole conoscere il numero dei customer in ciascuno Stato.

La query SQL che risolve tale problema è la seguente:

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country;
```

Un estratto del risultato dell'elaborazione della query è il seguente:

Result:

Number of Records: 21

COUNT(CustomerID)	Country
3	Argentina
2	Austria
2	Belgium
9	Brazil
3	Canada
2	Denmark

Una tabella può essere partizionata in gruppi a cui applicare una condizione per selezionarne solo alcuni. Questa selezione (filtro) viene realizzata impiegando la **clausola HAVING**.

La clausola HAVING può essere utilizzata insieme a una qualsiasi funzione di aggregazione, ma deve essere sempre associata ad una clausola GROUP BY. Differisce dal filtro attuato dal WHERE che, invece, non agisce sulla

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

partizione. Per esempio, si vuole conoscere il numero dei customer in ciascuno Stato, ma solo di quegli Stati che hanno almeno 5 Customer. La query SQL che risolve questo problema è la seguente:

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
HAVING COUNT(CustomerID)>=5;
```

Il risultato dell'elaborazione della query è il seguente:

Result:

Number of Records: 7

COUNT(CustomerID)	Country
9	Brazil
11	France
11	Germany
5	Mexico
5	Spain
7	UK
13	USA

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Query annidate

Le query annidate consentono di realizzare sotto-interrogazioni (**query interne**) annidate all'interno di altre query chiamate **esterne**. Le query annidate permettono di combinare insieme più SELECT per ottenere un unico risultato. Per eseguire due query in modo annidato (cioè una dentro l'altra) è necessario prima di tutto conoscere la sintassi generale delle SELECT nidificate:

- la subquery interna viene indicata tra parentesi tonde e viene eseguita per prima;
- sulla base del risultato della subquery, viene eseguita la query esterna.

Per esempio, si vogliono avere le informazioni del prodotto che ha il prezzo più alto. Per poter fare questo devo prima trovare il prezzo più alto tra i prodotti attraverso la seguente query:

```
SELECT MAX(Price)
FROM Products;
```

Successivamente devo interrogare la tabella Products per avere le informazioni del prodotto che presenta un prezzo uguale a quello massimo trovato in precedenza. La query annidata che risolve il problema è la seguente:

```
SELECT *
FROM Products
WHERE Price = (SELECT MAX(Price)
               FROM Products)
```

Il risultato dell'elaborazione della query precedente è il seguente:

Result:

Number of Records: 1

ProductID	ProductName	SupplierID	CategoryID	Unit	Price
38	Côte de Blaye	18	1	12 - 75 cl bottles	263.5

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Camagni, P., Nikolassy, R. (2018). Database SQL e PHP. Hoepli.
- W3schools (2021). https://www.w3schools.com/

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).